Programação Orientada aos Objetos
2026/2027
Projeto

POOGym

Prazo de entrega Fase 1: 25/10/2026
Prazo de entrega Fase 2: 06/12/2026
Plataforma: Inforestudante
Cotação: 7 valores (Fase 1: 3 valores; Fase 2: 4 valores)

Nota: A fraude denota uma grave falta de ética e constitui um comportamento não admissível. Situações de fraude serão severamente penalizadas, de acordo com os regulamentos e a legislação em vigor.

Um ginásio pretende uma aplicação, POOGym, para gerir as inscrições dos seus sócios em aulas de grupo e em sessões de *personal training*. Os sócios podem ser individuais ou ao abrigo de um protocolo empresarial, e cada tipo de sócio tem um desconto próprio. As atividades disponíveis podem ser aulas em grupo (em várias modalidades) ou sessões de *personal training* (em diferentes níveis).
Os sócios são caracterizados pelo nome, pelo número de sócio (único) e pelo e-mail. O sócio individual indica ainda o ano de nascimento, sendo aplicado um desconto de 10% aos sócios com menos de 18 anos e de 15% aos com 65 anos ou mais. O sócio empresa indica o nome e o NIF da empresa parceira, beneficiando de um desconto fixo de 20% em todas as marcações.
As atividades são caracterizadas pelo nome, pelo instrutor responsável, data e hora de início, e pela duração em minutos. O instrutor/*personal trainer* é caracterizado pelo nome, género e formação (texto com a lista de cursos e formações). A aula de grupo indica a modalidade (Yoga, Spinning, Zumba ou CrossFit) e a capacidade máxima de participantes. A sessão de *personal training* indica o nível do treino (Iniciação, Intermédio ou Avançado).
A tabela seguinte apresenta o custo base (em euros) de cada sessão.

Atividade / Nível	Custo base (Euro)
Aula de grupo — Yoga	8,00
Aula de grupo — Spinning	9,00
Aula de grupo — Zumba	8,50
Aula de grupo — CrossFit	12,00
Personal Training — Iniciação	18,00
Personal Training — Intermédio	22,00
Personal Training — Avançado	28,00

O projeto será desenvolvido em duas fases, com duas datas de entrega e defesas distintas.

Fase 1

Nesta fase do trabalho pretende-se que o aluno inicie o desenvolvimento da aplicação POOGym. Deverá ser entregue o diagrama de classes UML para esta fase do projeto e as classes necessárias (utilizando corretamente os conceitos de encapsulamento e herança). Deverá dar prioridade ao uso de coleções para o armazenamento de dados. Na fase 1 do

projeto, os alunos devem utilizar apenas os conceitos de POO dados nas aulas e não devem utilizar conceitos a abordar posteriormente, como por exemplo, polimorfismo, classes abstratas, ou ficheiros.

Funcionalidades a implementar:

1. **Registar sócio**: pedir os dados e criar um sócio individual ou um sócio empresa.
2. **Registar instrutor**: pedir os dados e criar um instrutor/*personal trainer*.
3. **Registar atividade**: criar uma aula de grupo ou uma sessão de *personal training*.
4. **Listar sócios**: apresentar todos os sócios com os respetivos dados.
5. **Listar atividades**: apresentar todas as atividades com os respetivos dados.
6. **Criar marcação**: associar um sócio a uma atividade e a uma data/hora, calculando e apresentando o custo final (com o desconto aplicável).
7. **Listar marcações**: apresentar todas as marcações, respetivos custos e total faturado.
8. **Resumo de atividades:** apresentar, para cada tipo de sócio e para cada tipo de atividade, o número de registos existentes (por exemplo: "Sócios individuais: 4, Sócios empresa: 3", "Aulas de grupo: 6, Sessões de PT: 3").
9. **Mais popular:** determinar e apresentar qual a atividade com mais marcações associadas.
10. **Valores Sócio:** pedir ao utilizador o número de um sócio e apresentar, para esse sócio, o valor total que já gastou no ginásio e o valor total que já poupou graças ao seu desconto, somando os valores de todas as suas marcações.

Fase 2

Nesta fase, deve-se evoluir a solução entregue na Fase 1, corrigindo e aprimorando o trabalho com base nos comentários e na avaliação daquela fase, e aplicando os conceitos abordados posteriormente na disciplina, incluindo polimorfismo, classes abstratas e ficheiros. Deverá rever o código e as funcionalidades entregues anteriormente para, quando adequado, utilizar os novos conceitos. Deverá criar o diagrama de classes UML final com as classes atualizadas. Deve passar a ser feita a leitura e escrita de dados em ficheiros de texto e de objetos, bem como o tratamento correto de exceções.
Funcionalidades adicionais a implementar:

11. **Carregar sócios**: ao iniciar, carregar os sócios a partir do ficheiro de texto `socios.txt`, caso exista.
12. **Carregar instrutores**: ao iniciar, carregar os instrutores a partir do ficheiro de texto `instrutores.txt`, caso exista.
13. **Carregar atividades**: ao iniciar, carregar as atividades a partir do ficheiro de texto `atividades.txt`, caso exista.
14. **Carregar marcações**: ao iniciar, carregar as marcações a partir de um ficheiro de objetos (`marcacoes.obj`), caso exista.
15. **Guardar marcações**: antes de terminar, exportar as marcações para o ficheiro de objetos, substituindo as que eventualmente existam (o ficheiro deve ser criando caso não exista).
16. **Ordenar marcações**: listar as marcações ordenadas por custo final, do maior para o menor.
17. **Valores Sócio:** reimplementar esta funcionalidade da Fase 1 que apresenta, para cada sócio, o valor total gasto e o valor total poupado em descontos. Compare, no relatório, a diferença de complexidade entre as duas soluções.

18. **Faturação:** apresentar o valor total faturado pelo ginásio, agrupado por categoria de atividade (por exemplo, aulas de grupo versus sessões de *personal training*).
19. **Mais poupado:** apresente o sócio que mais poupou, graça aos seus descontos, em valor absoluto.

Implementação

A aplicação deverá ser implementada na linguagem Java e ter em conta os seguintes aspetos:

a. Elaboração de um diagrama de classes (UML) antes de iniciar a implementação, para prever a estrutura do projeto. Na Fase 1, o diagrama não deve incluir polimorfismo nem classes abstratas; na Fase 2, o diagrama deve ser atualizado para refletir a evolução da solução.
b. Serão fortemente penalizadas soluções que não sigam os princípios da programação orientada a objetos, bem como os conceitos de herança e de polimorfismo.
c. A utilização de `instanceof`, `class.getName()` ou de métodos semelhantes será fortemente penalizada.
d. As soluções devem considerar boas práticas de programação.
e. Cada classe deverá gerir internamente os seus dados, pelo que deverá cuidar da proteção das suas variáveis e dos seus métodos.
f. Cada classe deverá ser responsável por uma tarefa ou objetivo específico, não lhe devendo ser atribuídas funções indevidas.
g. Utilize a `keyword static` apenas quando se justificar e não para contornar erros do compilador.
h. Comente as classes e os métodos públicos no formato `Javadoc`. Isto permitirá gerar automaticamente uma estrutura de ficheiros em HTML. Comente o restante do código sempre que a leitura dos algoritmos não for óbvia.
i. Será penalizado o uso abusivo de variáveis e métodos públicos.
j. Na escolha de nomes para variáveis, classes e métodos, devem ser seguidas as convenções adotadas na linguagem Java.
k. Na organização das classes, deve-se evitar a redundância do código.

A aplicação a desenvolver deve ser disponibilizada, nas duas fases, com dados de demonstração (pelo menos: seis sócios - três de cada tipo -, oito atividades e seis marcações). Na Fase 1, estes dados podem ser criados diretamente no código, com dados fixos (por exemplo, no arranque da aplicação, num método **`void carregarDados()`**); na Fase 2, estes mesmos dados devem passar a ser carregados a partir dos ficheiros de texto (`socios.txt`, `atividades.txt`) e de objetos (`marcacoes.obj`), conforme descrito anteriormente. A estrutura dos ficheiros de texto é definida pelos estudantes, tendo em conta a facilidade de edição e a simplificação do *parsing* pela aplicação.

Utilização de Inteligência Artificial

A utilização de Inteligência Artificial (IA) generativa (ex.: ChatGPT, Claude, Gemini, etc.) para suporte à realização do projeto da cadeira de POO não é recomendada e, caso os alunos recorram a alguma ferramenta, ela deve ser usada de forma crítica. O nível de utilização é: Uso de forma construtiva.

A IA é, atualmente, um elemento incontornável do dia a dia de um programador. A utilização consciente e crítica destes instrumentos é uma mais-valia. A sua utilização deve ser feita de forma construtiva e consciente das suas limitações (ex.: alucinações). A sua utilização é recomendada para situações como, por exemplo, esclarecimentos conceituais, suporte ao *debugging* e "inline copilot" / autocomplete (sintaxe). A sua utilização não é recomendada em situações como, por exemplo, o uso de um *prompt* para gerar soluções para problemas.

No âmbito desta cadeira, os alunos ainda não têm competências suficientes para interpretar criticamente a solução apresentada pelo modelo de IA, comprometendo a resolução correta do projeto e o respetivo objetivo pedagógico. Compromete ainda a defesa e avaliação do projeto.

Entrega

O projeto deve ser realizado em grupos de dois estudantes da mesma turma prática e é avaliado em duas fases, cada uma com entrega no Inforestudante.

Em ambas as fases devem ser entregues os seguintes ficheiros:

- Diagrama de classes em UML (pdf).
- Todas as classes .java (sem compressão).
- Ficheiros .txt e .obj de suporte à execução do programa (apenas na Fase 2).
- Ficheiro ZIP com o Javadoc (ZIP).
- Relatório (pdf).

Avaliação do trabalho

A avaliação do projeto está dividida em duas fases, sendo, em cada uma, considerados fatores de dois tipos:

- **Caixa preta** (tal como é percecionada pelo utilizador):
    - Conjunto de funcionalidades implementadas;
    - Robustez do programa;
    - Qualidade da interface.
- **Caixa branca** (a forma como está construído):
    - Qualidade das soluções técnicas encontradas para os problemas;
    - Estruturação do código;
    - Qualidade dos comentários;
    - Aplicação dos conceitos da matéria relevantes para cada fase (ver secção acima).

Nota: Não se aceitam trabalhos que apresentem erros de compilação no momento da defesa.

Defesa do trabalho

O trabalho terá duas defesas, uma por fase, por meio de uma discussão presencial, nas duas semanas seguintes ao fim do prazo de entrega de cada fase. Os alunos deverão inscrever-se, no **Inforestudante**, num horário de defesa para cada uma das fases.

As defesas serão divididas em duas etapas: na primeira, os alunos serão questionados sobre o trabalho entregue e a matéria da disciplina; na segunda, serão propostos alguns desafios para desenvolver novas funcionalidades ou alterar funcionalidades existentes no código entregue.

MUITO IMPORTANTE

Os trabalhos serão comparados (tanto entre os trabalhos da disciplina – todas as turmas - como com o código disponível na Internet), no sentido de detetar eventuais fraudes por cópia. Nos casos em que se verifique que houve cópia de trabalho total ou parcial, os grupos envolvidos terão os projetos anulados, reprovando à disciplina. Serão aplicadas as regras da Universidade de Coimbra relativas ao plágio.